

108. La vitesse : un pas toutes les N images

Un pixel par image, c'est 60 pixels par seconde. Pour aller moins vite, on n'avance pas à chaque image.

1. Ce qu'il faut comprendre

La leçon d'avant avançait d'un pixel à **chaque image** : 60 pixels par seconde, l'écran traversé en moins de trois secondes. C'est vif. Un personnage qui marche lentement, une tortue, un nuage, doivent aller **moins vite**.

On ne peut pas avancer d'un demi-pixel. Alors on **saute des images** : on n'avance qu'une image sur N. Une sur 2 : 30 pixels par seconde. Une sur 4 : 15.

Ce qui est nouveau ici : `lenteur`, une variable qui règle la vitesse. `attente` compte les images ; quand elle atteint `lenteur`, on la remet à 0 et on fait **un pas**. C'est la montre du chapitre 3 (« une boucle qui dure plusieurs images »), mise au service du mouvement.

Déroulons avec `lenteur = 4` : image 1, `attente` vaut 1 → rien. Image 2 → 2, rien. Image 3 → 3, rien. Image 4 → 4 : **un pas**, et `attente` revient à 0. Un pas toutes les 4 images.

A règle `lenteur` à 1 (le plus rapide : un pas par image), B à 4 (quatre fois plus lent). Pour aller **plus vite** qu'un pixel par image, on ferait `x = x + 2` : deux pixels d'un coup. Au-delà de 3 ou 4 pixels, l'œil recommence à voir des sauts.

À toi : donne à GAUCHE et DROITE deux lenteurs différentes — un personnage qui recule plus lentement qu'il n'avance.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **743 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	68 o	← la console, à l'allumage — la mise en route de la console
main()	136 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide

```

CopierTuiles      130 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            224 o ← CopierTuiles – 13 lettres sur 43, et 1 tuile dessinée
DonneesTransfert  10 o ← InstallerTransfert – la petite routine qui copie les lutins
"A: RAPIDE 1"     12 o ← texte() – un texte gravé
"B: LENT 4"       12 o ← texte() – un texte gravé

```

Lettres gravées : ABDEILNPRT14:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite() × 1    23 o sur place, dans main()
texte() × 2     22 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 4    20 o sur place, dans main() + LireManette (37 o)
image() × 1     6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : GAUCHE / DROITE déplacent un héros ; A le rend rapide,
// B le rend lent.
//
// Ce qui est nouveau : régler la VITESSE. On ne peut pas avancer d'un demi-pixel ;
// alors on n'avance qu'une image sur « lenteur ».
//  lenteur = 1 : un pas par image          → 60 pixels par seconde
//  lenteur = 4 : un pas toutes les 4 images → 15 pixels par seconde

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près

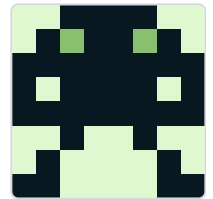
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {                // . = transparent, - = clair, # = sombre
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};

```



à l'écran

3. UNE VARIABLE — X

```
uint8_t x = 20;    // la position, en pixels
```

4. UNE VARIABLE — LENTEUR

```
uint8_t lenteur = 4;    // un pas toutes les 4 images, au départ
```

5. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0;    // les images comptées depuis le dernier pas
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(1, 1, "A: RAPIDE  1");
    texte(1, 2, "B: LENT   4");

    while (true) {                          // La boucle du jeu :
        image();

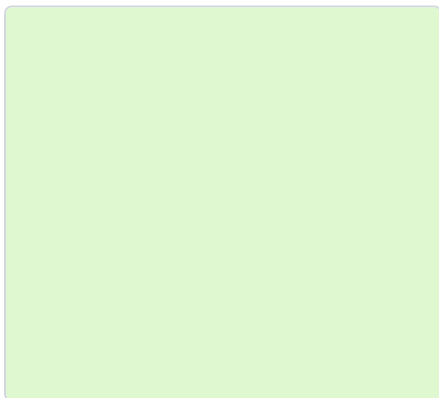
        if (bouton(A)) lenteur = 1;         // un pas à chaque image : 60 pixels par seconde
        if (bouton(B)) lenteur = 4;         // un pas toutes les 4 images : 15 par seconde

        attente++;                         // une image de plus
        if (attente >= lenteur) {           // c'est le moment d'un pas ?
            // Avec lenteur = 4 : images 1, 2, 3 → non ; 4 → oui.
            attente = 0;                   // oui : on recommence à compter
            if (bouton(DROITE) && x < 152) x++; // un pixel à droite
            if (bouton(GAUCHE) && x > 0) x--;  // un pixel à gauche
        }

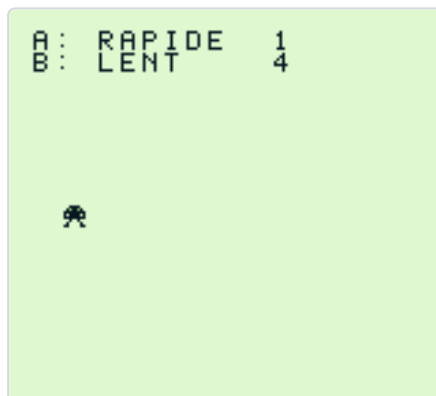
        sprite(0, x, 72, HEROS);           // On pose le héros à sa place, à chaque image.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



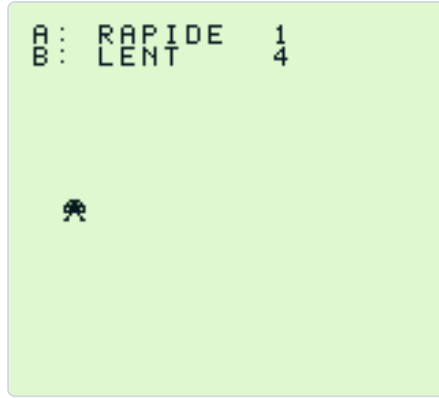
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

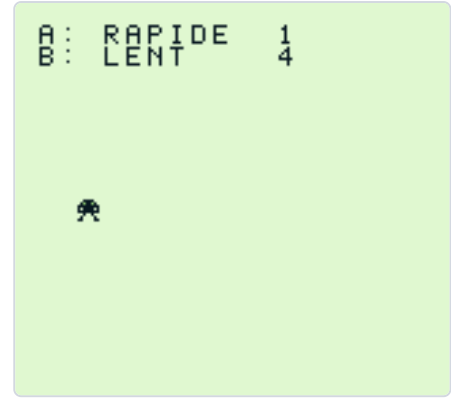
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



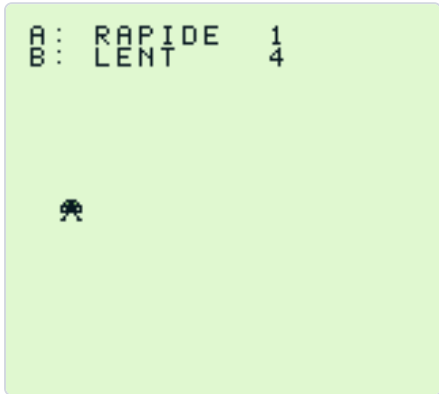
A — l'écran n'a pas bougé



B — l'écran n'a pas bougé



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un héros que DROITE et GAUCHE déplacent lentement ; après A, il file quatre fois plus vite ; B le ralentit de nouveau.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ lent : environ un pixel toutes les 4 images (10 pixels en 40 images)
- ✓ après A : un pixel par image (40 pixels en 40 images)
- ✓ B le ralentit de nouveau (10)

Ensuite : **109. Animer : deux dessins pour marcher** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++